



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Παράλληλα Συστήματα
Εργασία Simd

Ονοματεπώνυμο: Ευάγγελος Τραϊκόγλου

Αριθμός Μητρώου: 1115202200189

- cpu: i7-8700
- cores: 12 logical, 6 physical
- os, kernel: Ubuntu 24.04.3 LTS, 6.8.0-94-generic
- comp version: gcc 13.3.0

1 Polynomial Multiplication

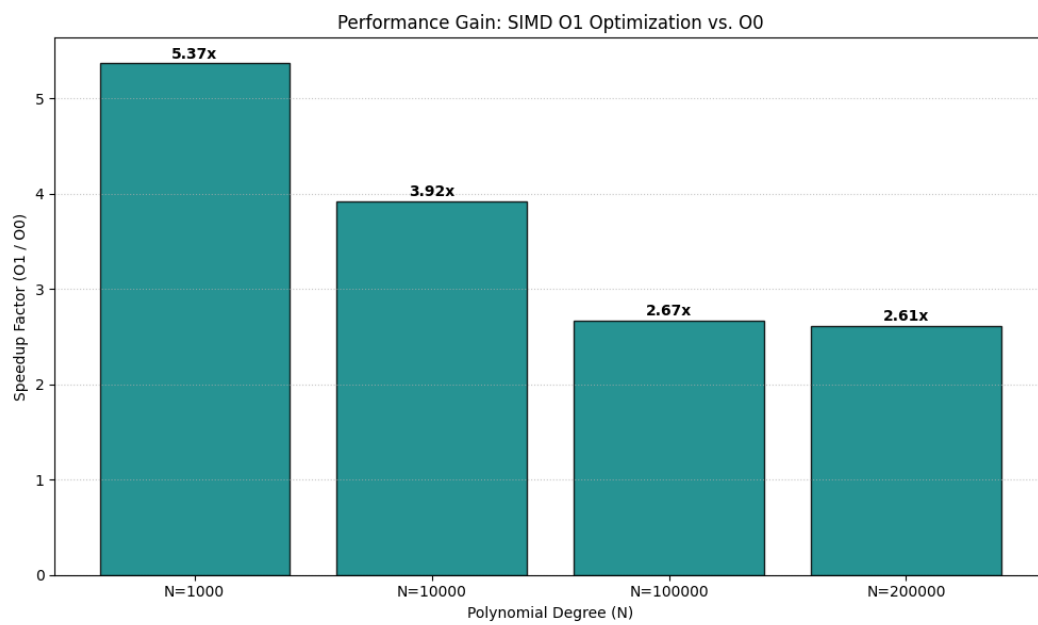
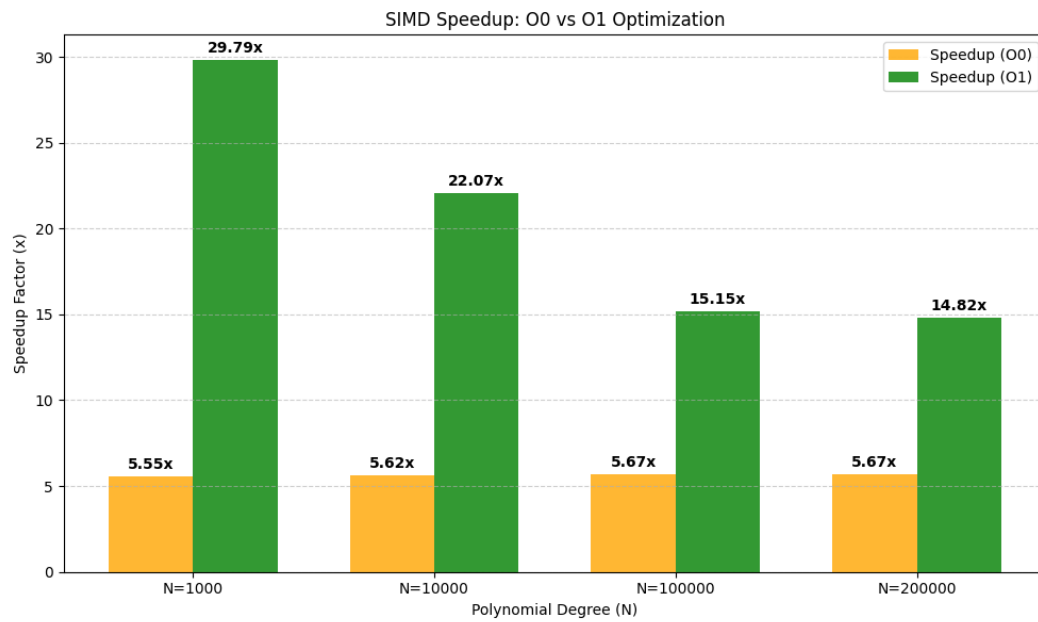
Η άσκηση επικεντρώνεται στον πολλαπλασιασμό πολυωνύμων βαθμού n . Ο σειριακός αλγόριθμος ακολουθεί την απλή υλοποίηση $O(n^2)$ χρόνου. Για την παράλληλη εκδοχή υπάρχουν 2 υλοποιήσεις γιατί παρατηρήθηκε ότι με τα optimize flags off κάθε update που γινόταν σε 1 register είχε ως αποτέλεσμα την άμεση καταγραφή και στο stack παρόλο που δεν χρειαζόταν αφού θα χρησιμοποιούνταν αμέσως μετά. Αυτό ήταν το κύριο bottleneck. Απ' ότι διάβασα είναι για λόγους debugging. Οπότε υπάρχει και 2 υλοποίηση (simd_mul_O1) που χρησιμοποιεί το O1 flag και η διαφορά στο χρόνο είναι σημαντική: $\sim 2.3\text{sec}$, 100k degree input. Στην simd_mul_O1, χρησιμοποιούνται simd εντολές 256bit/8 elements (load,store,add,multiply). Ακόμη ο βρόχος ξετυλίγεται (4 επαναλήψεις, saves 2.2sec on 100k degree input).

Σημείωση: Το base για τα optimizations που εξηγούνται παρακάτω είναι η simd_mul_O1 χωρίς το directive για optimize.

Στην simd_mul έγιναν κάποιες βελτιστοποιήσεις με το χέρι. Όπως ειπώθηκε το βασικό πρόβλημα είναι τα αχρείαστα store στο stack. Για αυτό το λόγο προστέθηκε το directive ACCUMULATE_8 που εμφωλευμένα καλεί όλα τα calls που πρέπει να γίνουν για την δάδα στοιχείων (load,mult,add,store). Αυτό εξαλείφει μερικά stack writes και ρίχνει τον χρόνο ($\sim 0.6\text{sec}$ για 100k degree pol). Ακόμα το pointer arithmetic για να βρεθεί η επόμενη θέση έναντι του απλού access, eg: $a[i+j]$ ρίχνει τον χρόνο κατά 1.3sec, 100k pol-deg. Τέλος, το prefetch του επόμενου cache line ρίχνει το χρόνο κατά: $\sim 0.2\text{sec}$ 100k, $\sim 1\text{sec}$ 200k.

Run: `./polynomial_mult -n <degree>`

Σημείωση: Όλα τα πειράματα υπάρχουν στο csv αρχείο: simd_bench_results.csv.



Συμπεράσματα: Η επιτάχυνση που παρατηρείται είναι σταθερή (~ 5.6) για όλα τα μεγέθη πολυωνύμων αφού χρησιμοποιείται το ίδιο μοτίβο γενικά της επεξεργασίας 8 στοιχείων τη φορά μέσω vector registers. Το βασικό bottleneck στη περίπτωση του `simd_mul` είναι το memory traffic λόγω των αχρείαστων stack writes που αν και μειώθηκε λόγω των βελτιστοποιήσεων, δεν εξαφανίστηκε πλήρως. Προφανώς στη περίπτωση O1 και άλλες βελτιστοποιήσεις μπορεί να βοηθάνε συμπληρωματικά (πχ καλύτερο scheduling εντολών) για αυτό και παρατηρείται μεγάλη διαφορά μεταξύ των 2 υλοποιήσεων (κυρίως στα μικρά μεγέθη), ενώ σταθεροποιείται περίπου στο ~ 2.5 η επιτάχυνση O1 έναντι του O0 στα μεγαλύτερα.

2 Optimize Maxwell's Equations

Το πρόβλημα αφορά την προσομοίωση της διάδοσης ηλεκτρομαγνητικών κυμάτων σε ένα δισδιάστατο πλέγμα χρησιμοποιώντας τις εξισώσεις του Maxwell.

Part 1, 2

Οι update συναρτήσεις χρησιμοποιούν zip iterators & gpu transforms, για κάθε μετασχηματισμό που χρειάζεται. Συμπληρώθηκαν `__device__`, `thrust::device` όπου χρειαζόταν (πχ transform). Τέλος, η `update_dz` έχει 2 counting iterators. Οι temp buffers που υπήρχαν αντικαταστήθηκαν με τους iterators.

```

1 %%writefile Sources/maxwell.cu
2 #include "dli.h"
3 #include <thrust/device_vector.h>
4 #include <thrust/transform.h>
5
6 void update_hx(int n, float dx, float dy, float dt, thrust::device_vector<float> &hx,
7               thrust::device_vector<float> &ez) {
8     auto zip_begin = thrust::make_zip_iterator(
9         thrust::make_tuple(ez.begin() + n, ez.begin()));
10
11     auto diff_begin = thrust::make_transform_iterator(
12         zip_begin,
13         [] __device__ (thrust::tuple<float, float> t) -> float {
14             return thrust::get<0>(t) - thrust::get<1>(t);
15         });
16
17     thrust::transform(
18         thrust::device,
19         hx.begin(),
20         hx.end() - n,
21         diff_begin,
22         hx.begin(),
23         [dt, dx, dy] __device__ (float h, float cex) -> float {
24             return h - dli::C0 * dt / 1.3f * cex / dy;
25         });
26 }
27
28 void update_hy(int n, float dx, float dy, float dt, thrust::device_vector<float> &hy,
29               thrust::device_vector<float> &ez) {
30     auto zip_begin = thrust::make_zip_iterator(
31         thrust::make_tuple(ez.begin(), ez.begin() + 1));
32
33     auto diff_begin = thrust::make_transform_iterator(
34         zip_begin,
35         [] __device__ (thrust::tuple<float, float> t) -> float {
36             return thrust::get<0>(t) - thrust::get<1>(t);

```

```

37     });
38
39     thrust::transform(
40         thrust::device,
41         hy.begin(),
42         hy.end() - 1,
43         diff_begin,
44         hy.begin(),
45         [dt, dx, dy] __device__ (float h, float cey) -> float {
46             return h - dli::C0 * dt / 1.3f * cey / dx;
47         });
48 }
49
50 void update_dz(int n, float dx, float dy, float dt, thrust::device_vector<float> &hx_vec,
51               thrust::device_vector<float> &hy_vec, thrust::device_vector<float> &dz_vec) {
52
53     float* hx_ptr = thrust::raw_pointer_cast(hx_vec.data());
54     float* hy_ptr = thrust::raw_pointer_cast(hy_vec.data());
55     float* dz_ptr = thrust::raw_pointer_cast(dz_vec.data());
56
57     auto hx = hx_vec.begin();
58     auto hy = hy_vec.begin();
59     auto dz = dz_vec.begin();
60     thrust::for_each(thrust::device,
61                     thrust::make_counting_iterator<int>(0),
62                     thrust::make_counting_iterator<int>(n*n),
63                     [=] __device__ (int cell_id) {
64                         if (cell_id > n) {
65                             float hx_diff = hx_ptr[cell_id - n] - hx_ptr[cell_id];
66                             float hy_diff = hy_ptr[cell_id] - hy_ptr[cell_id - 1];
67                             dz[cell_id] += dli::C0 * dt * (hx_diff / dx + hy_diff / dy);
68                         }
69                     });
70 }
71
72 void update_ez(thrust::device_vector<float> &ez, thrust::device_vector<float> &dz) {
73     thrust::transform(thrust::device, dz.begin(), dz.end(), ez.begin(),
74                     [=] __device__ (float d) -> float {return d / 1.3f; });
75 }
76
77 void simulate(int cells_along_dimension, float dx, float dy, float dt,
78              thrust::device_vector<float> &d_hx,
79              thrust::device_vector<float> &d_hy,
80              thrust::device_vector<float> &d_dz,
81              thrust::device_vector<float> &d_ez) {
82     for (int step = 0; step < dli::steps; step++) {
83         update_hx(cells_along_dimension, dx, dy, dt, d_hx, d_ez);
84         update_hy(cells_along_dimension, dx, dy, dt, d_hy, d_ez);
85         update_dz(cells_along_dimension, dx, dy, dt, d_hx, d_hy, d_dz);
86         update_ez(d_ez, d_dz);
87     }
88 }

```

```
Once you address all FIXME (Step 1) comments, run the code cell below to see how your changes affect performance. The result will automatically be recorded for the autograder.

[45]: # Do not change this cell; results are recorded for the autograder
Sources.dli.passes_step_1_with_n_of(1024)

Running Maxwell simulator with 1024^2 cells
-----

Verify performance:
  achieved throughput: 4308387340 cells / second
  expected throughput: 1933580719 cells / second
  solution is fast enough

Verify correctness
  solution is correct
```

Figure 1: Part 1 results

```
[49]: # Do not change this cell; results are recorded for the autograder
Sources.dli.passes_step_2_with_n_of(1024)

Running Maxwell simulator with 1024^2 cells
-----

Verify performance:
  achieved throughput: 4305998876 cells / second
  expected throughput: 3651013643 cells / second
  solution is fast enough

Verify correctness
  solution is correct
```

Figure 2: Part 2 results

Part 3

Το πλέγμα χωρίστηκε σε μικρότερα tiles, όπου κάθε thread block αναλαμβάνει έναν “coarse” tile και κάθε thread μέσα στο block χειρίζεται ένα στοιχείο του fine grid. Το BlockReduce συγκεντρώνει το άθροισμα στο thread 0 του block. Η μέση τιμή υπολογίζεται με διαίρεση του αθροίσματος με τον αριθμό των στοιχείων στο tile. Η εγγραφή της τιμής στο coarse grid γίνεται μόνο από thread 0, εξασφαλίζοντας ότι δεν δημιουργούνται data races.

```
1 %%writefile Sources/coarse.cu
2 #include "dli.h"
3
4 __global__ void kernel(dli::temperature_grid_f fine,
5                       dli::temperature_grid_f coarse) {
6     int coarse_row = blockIdx.x / coarse.extent(1);
7     int coarse_col = blockIdx.x % coarse.extent(1);
8     int row = threadIdx.x / dli::tile_size;
9     int col = threadIdx.x % dli::tile_size;
10    int fine_row = coarse_row * dli::tile_size + row;
11    int fine_col = coarse_col * dli::tile_size + col;
12
13    float thread_value = fine(fine_row, fine_col);
14
15    using BlockReduce = cub::BlockReduce<float, dli::block_threads>;
16    __shared__ typename BlockReduce::TempStorage temp_storage;
17    float block_sum = BlockReduce(temp_storage).Sum(thread_value);
18
19    if(threadIdx.x == 0){
```

```

20     float block_average = block_sum / (dli::tile_size * dli::tile_size);
21     coarse(coarse_row, coarse_col) = block_average;
22 }
23 }
24
25 void coarse(dli::temperature_grid_f fine, dli::temperature_grid_f coarse) {
26     kernel<<<coarse.size(), dli::block_threads>>>(fine, coarse);
27 }

```

```

[52]: # Do not change this cell; results are recorded for the autograder
Sources.dli.passes_step_3_with_n_of(1024)

Running Maxwell simulator with 1024^2 cells
-----

Verify performance:
    achieved throughput: 4308890669 cells / second
    expected throughput: 3599378874 cells / second
    solution is fast enough

Verify correctness
    solution is correct

[52]: True

```

Figure 3: Part 3 results



Figure 4: Certificate

Grade summary ⓘ

Assignment type	Weight	Grade	Weighted grade
GPU Task	100%	100%	100%
Your current weighted grade summary			100%

Figure 5: Grade